

Audyt techniczny integracji

zdrovena-reconciliation — 5 integracji zewnętrznych

Repozytorium	github.com/PiotrGry/zdrovena-reconciliation
Branch	develop @ 39ad39b
Data audytu	02.07.2026
Zakres	Allegro · Fakturownia · Shopify · InPost · Apaczka
Tryb	ruff strict + pyright strict + manual review
Autor	Perplexity Computer (dla Piotra Gryzlo, HUMIO)

Werdykt: Zidentyfikowano **10 usterek krytycznych** (5 Allegro + 3 Shopify + 2 antypatery w InPost/Apaczka) do naprawy w **jednym gigantycznym PR**. Estymata: ~1400 linii kodu + ~800 linii testów, 11 commitów.

10 Krytyczne	76 Antypatery ruff	0 Błędy pyright	5 Integracji
-----------------	-----------------------	--------------------	-----------------

Spis treści

00 · Streszczenie wykonawcze

01 · Wysyłam z Allegro

5 KRYT

02 · Fakturownia

OK

03 · Shopify Webhooks

3 KRYT

04 · InPost ShipX

1 KRYT antypattern

05 · Apaczka

1 KRYT antypattern

06 · Antypatrzny wspólny + Plan PR

Audyt techniczny integracji — Executive Summary

Data: 2026-07-02 **Zakres:** Allegro, Fakturownia, Shopify, InPost, Apaczka **Baseline:** `develop` @ `39ad39b` (post-PR#75) **Autor:** Perplexity Computer

1. Metoda audytu

Dla każdej integracji sprawdzamy **8 wymiarów**:

1. **URL + method** — czy zgadza się z oficjalną dokumentacją API
2. **Request fields** — nazwy, typy, wartości enumów, wymagalność
3. **Response fields** — nazwy pól z których czytamy dane (tu żyje bug Allegro `invoiceNumber`)
4. **Headers** — Content-Type, Accept, Authorization
5. **Antypaterny** — `except Exception`, magic strings, brak type hints, duplikacja logiki
6. **Idempotency** — czy retry/duplicate call nie stworzy duplikatu przesyłki/faktury
7. **Exception handling** — czy używamy typowanej hierarchii `ZdrovenaShippingError` czy `raise RuntimeError`
8. **Logging** — czy każdy call ma log, czy błędy mają `exc_info=True`, czy komunikaty są czytelne

Tryb pracy: B (ruff `--select ALL` + pyright strict + manual review, per file). **Format wyników:** C (Markdown + PDF na koniec).

2. Inwentaryzacja endpointów

2.1 Allegro (`zdrovena/common/allegro.py`)

Base URL: `https://api.allegro.pl` (prod) | `https://api.allegro.pl.allegrosandbox.pl` (sandbox)
Auth: OAuth 2.0 refresh-token → `Bearer <access>` (12h żywotności) Content-Type: `application/vnd.allegro.public.v1+json`

Metoda klienta	HTTP	Path	Grupa	Wołane w
<code>_fetch_token</code>	POST	<code>/auth/oauth/token</code> (Basic)	OAuth	<code>_get_token</code>
<code>list_orders</code>	GET	<code>/order/checkout-forms</code>	Orders	<code>allegro_poller.poll_orders_once</code>
<code>get_order</code>	GET	<code>/order/checkout-forms/{id}</code>	Orders	(nieużywane)
<code>mark_order_processed</code>	PUT	<code>/order/checkout-forms/{id}/fulfillment</code>	Orders	<code>allegro_poller</code> (za flagą <code>ALLEGRO_MARK_ON_DRAFT</code>)

Metoda klienta	HTTP	Path	Grupa	Wołane w
<code>create_shipment</code>	POST	<code>/order/checkout-forms/{id}/shipments</code>	Tracking push	<code>webhooks._maybe_push_tracking_to_allegro</code>
<code>get_shipments</code>	GET	<code>/order/checkout-forms/{id}/shipments</code>	Tracking push	<i>(nieużywane)</i>
<code>list_order_invoices</code>	GET	<code>/order/checkout-forms/{id}/invoices</code>	Invoices	<code>fakturownia_patcher.patch_allegro_invoices_once</code>
<code>create_invoice_declaration</code>	POST	<code>/order/checkout-forms/{id}/invoices</code>	Invoices	<i>(nieużywane — placeholder do przyszłego upload)</i>
<code>upload_invoice_file</code>	PUT	<code>/order/checkout-forms/{id}/invoices/{invId}/file</code>	Invoices	<i>(nieużywane)</i>
<code>get_delivery_services</code>	GET	<code>/shipment-management/delivery-services</code>	Wysyłam z Allegro	<i>(bootstrap — nieużywane w runtime)</i>
<code>get_delivery_proposal</code>	GET	<code>/shipment-management/delivery-proposals/{id}</code>	Wysyłam z Allegro	<i>(nieużywane)</i>
<code>create_ship_with_allegro_shipment</code>	POST	<code>/shipment-management/shipments/create-commands</code>	Wysyłam z Allegro	<code>webhooks._run_allegro_delivery</code>
<code>get_ship_with_allegro_command_status</code>	GET	<code>/shipment-management/shipments/create-commands/{cmd}</code>	Wysyłam z Allegro	<code>wait_for_ship_with_allegro_shipment</code> (loop)
<code>get_ship_with_allegro_shipment</code>	GET	<code>/shipment-management/shipments/{shipId}</code>	Wysyłam z Allegro	<code>webhooks._run_allegro_delivery</code>
<code>get_ship_with_allegro_pickup_proposals</code>	POST	<code>/shipment-management/pickup-proposals</code>	Wysyłam z Allegro	<code>webhooks._run_allegro_delivery</code> (jeśli <code>pickup_date</code>)
<code>create_ship_with_allegro_pickup</code>	POST	<code>/shipment-management/pickups/create-commands</code>	Wysyłam z Allegro	<code>webhooks._run_allegro_delivery</code>
<code>get_ship_with_allegro_label</code>	GET	<code>/shipment-management/shipments/{shipId}/label</code>	Wysyłam z Allegro	<i>(nieużywane w routerze — do dodania label endpoint dla <code>allegro_delivery</code>)</i>

Endpointów: 16 (10 użytkowanych w produkcji, 6 nieużywanych).

2.2 Fakturownia (`zdrovena/common/fakturownia.py`)

Base URL: `https://zdrovena.fakturownia.pl` (z env `FAKTUROWNIA_BASE_URL`) Auth: `api_token` w query string + w body dla PUT/POST Content-Type: `application/json` (implicit z `json=` w requests)

Metoda klienta	HTTP	Path	Wołane w
<code>get_invoice</code>	GET	<code>/invoices/{id}.json</code>	<code>add_settlement_position</code> , <code>fakturownia_patcher._process_one_invoice</code>
<code>list_invoices</code>	GET	<code>/invoices.json?period=&page=&per_page=&number=&oid=&include_positions=</code>	<code>fakturownia_patcher._process_one_invoice</code>
<code>update_invoice</code>	PUT	<code>/invoices/{id}.json</code>	<code>add_settlement_position</code> (wewnątrz)
<code>add_settlement_position</code>	(2 calls: GET + PUT)	—	<code>fakturownia_patcher._process_one_invoice</code>

Endpointów: 3 (GET/PUT invoice, GET list) — plus composite `add_settlement_position`.

2.3 InPost (`zdrovena/common/inpost.py`)

Base URL: `https://api-shipx-pl.easypack24.net` (z env `INPOST_BASE_URL`) Auth: `Authorization: Bearer <token>` (session-level) Content-Type: `application/json`

Metoda klienta	HTTP	Path	Wołane w
<code>create_paczkomat_shipment</code>	POST	<code>/v1/organizations/{orgId}/shipments</code>	<code>webhooks._run_inpost</code> (paczkomat)
<code>create_kurier_shipment</code>	POST	<code>/v1/organizations/{orgId}/shipments</code>	<code>webhooks._run_inpost</code> (kurier)
<code>create_dispatch_order</code>	POST	<code>/v1/organizations/{orgId}/dispatch_orders</code>	<code>_run_inpost</code> + <code>webhooks.order_pickup</code>
<code>cancel_shipment</code>	DELETE	<code>/v1/shipments/{shipId}</code>	(nieużywane w routerze — brak endpointu DELETE)
<code>cancel_dispatch_order</code>	DELETE	<code>/v1/organizations/{orgId}/dispatch_orders/{disId}</code>	(nieużywane w routerze)
<code>get_label</code>	GET	<code>/v1/shipments/{shipId}/label</code>	<code>webhooks.get_label</code>

Endpointów: 4 (użytkowane) + 2 (cancel — martwy kod).

2.4 Apaczka (`zdrovena/common/apaczka.py`)

Base URL: `https://www.apaczka.pl/api/v2` Auth: HMAC-SHA256 signature per-request (`app_id`, `signature`, `expires`) Content-Type: `application/x-www-form-urlencoded` (implicit z `data=` w requests)

Metoda klienta	HTTP	Endpoint	Wołane w
<code>_get_service_structure</code>	POST	<code>service_structure</code>	<code>create_shipment</code> (przez cache 23h)
<code>create_shipment</code>	POST	<code>order_send</code>	<code>webhooks._run_apaczka</code>
<code>cancel_shipment</code>	POST	<code>order_cancel</code>	(nieużywane w routerze)

Metoda klienta	HTTP	Endpoint	Wołane w
<code>get_label</code>	POST	<code>waybill</code>	<code>webhooks.get_label</code>

Endpointów: 3 (użytkowane) + 1 (cancel — martwy kod).

2.5 Shopify (webhook przychodzący, nie klient)

HTTP	Path	Handler	Idempotency
POST	<code>/webhooks/shopify/order-created</code>	<code>shopify_order_created</code>	Dedup po <code>shopify_order_id</code> w <code>shipping_store</code> + HMAC-SHA256 verify

Uwaga: Shopify jest **konsumentem** (odbieramy webhook, nie wołamy API), więc audyt endpointów sprowadza się do jednego callback URL.

2.6 Router `webhooks.py` — publiczne endpointy naszego API

HTTP	Path	Handler	Auth role
POST	<code>/webhooks/shopify/order-created</code>	<code>shopify_order_created</code>	HMAC only
GET	<code>/shipping/drafts</code>	<code>list_drafts</code>	viewer+
POST	<code>/shipping/drafts/{id}/execute</code>	<code>execute_draft</code>	shipment_mgr+
POST	<code>/shipping/drafts/{id}/pickup</code>	<code>order_pickup</code>	shipment_mgr+
PATCH	<code>/shipping/drafts/{id}</code>	<code>update_draft</code>	shipment_mgr+
GET	<code>/shipping/drafts/{id}/label</code>	<code>get_label</code>	viewer+

! Rozjazd docstring vs kod: docstring modułu `webhooks.py` (linie 1-11) reklamuje 2 endpointy DELETE (`/shipment`, `/dispatch`), które **nie istnieją** w kodzie. Do usunięcia z docstringu albo do zaimplementowania — decyzja użytkownika.

3. Top-10 najpilniejszych obserwacji (wstępne, przed pełnym audytem)

#	Priorytet	Miejsce	Problem	Skutek
1	CRIT CRITICAL	<code>fakturownia_patcher.py:119</code>	<code>allegro_inv.get("number")</code> — Allegro API zwraca pole <code>invoiceNumber</code> (docs)	Patcher NIGDY nie dopisze kaucji na produkcji
2	CRIT CRITICAL	<code>webhooks.py</code> docstring l.1-11	Reklamuje endpointy DELETE które nie istnieją	Mylące — konsumenci API myślą że można anulować

#	Priorytet	Miejsce	Problem	Skutek
3	HIGH HIGH	<code>allegro_poller.py:56</code> <code>, 68, 87</code>	Trzy <code>except Exception</code> bez <code>exc_info=True</code>	Debug niemożliwy — nie widać stacktrace ani typu wyjątku
4	HIGH HIGH	<code>fakturownia_patcher.py:100, 113, 132</code>	<code>except Exception</code> łyka wszystko, w tym <code>KeyboardInterrupt</code> , <code>SystemExit</code> (przez <code>try/except Exception</code> , ale wciąż łyka <code>FakturowniaAuthError</code> bez odróżnienia)	Auth error traktowany jak business error → alert nie leci
5	HIGH HIGH	<code>webhooks._maybe_push_tracking_to_allegro:187</code>	<code>except Exception as exc</code> łyka wszystko	Auth Allegro fail → tylko warning, nikt się nie dowie
6	MED MEDIUM	<code>inpost.py:107</code>	<code>class InPostError(Exception)</code> — własna hierarchia zamiast <code>CourierBusinessError / CourierServerError</code>	Router nie odróżni 401 od 5xx od 422 — wszystko idzie do jednego <code>except</code>
7	MED MEDIUM	<code>apaczka.py:28</code>	Analogicznie: <code>class ApaczkaError(Exception)</code> — nie dziedziczy z <code>ZdrovenaShippingError</code>	Utrudnia unified handling
8	MED MEDIUM	<code>webhooks.py:869</code>	<code>except Exception as exc:</code> w <code>execute_draft</code> łyka wszystko , w tym <code>HTTPException</code>	Skutek niegroźny (i tak lecimy w 502) ale ukrywa root cause
9	MED MEDIUM	<code>allegro.py:229-236</code> (<code>mark_order_processed</code>)	Zawsze status= <code>PROCESSING</code> — brak weryfikacji czy zamówienie już nie jest w tym stanie (idempotent, ale marnujemy call)	Rate-limit ryzyko przy retry burst
10	MED MEDIUM	<code>webhooks.py:41</code>	<code>_MOCK_COURIER</code> czytane raz na import — nie da się zmienić bez restartu	Utrudnia testy manualne

UWAGA: to lista wstępna z 2h przeglądu. Pełny audyt (fazy 2-6 per integracja) może wygenerować więcej znalezisk.

4. Plan raportów per integracja

```
docs/audit/
├── 00-summary.md      ← ten plik
├── 01-allegro.md      ← pełny audyt Allegro (8 wymiarów)
```

```

├─ 02-fakturownia.md
├─ 03-inpost.md
├─ 04-shopify.md
├─ 05-apaczka.md
├─ 06-cross-cutting.md    ← wspólne antypatery, hierarchia wyjątków, standard logów
└─ fixtures/              ← realne JSON z docs dla contract tests

```

Każdy plik `0X-<integracja>.md` będzie miał sekcje:

1. **Endpoint contract table** (URL/method/req/resp/headers)
2. **Antypatery** (z linkami do linii kodu)
3. **Idempotency** (per operacja: safe/unsafe + rekomendacja)
4. **Exception handling** (mapa: który wyjątek → jaki HTTP status → jaka reakcja operatora)
5. **Logging** (co logujemy, na jakim poziomie, czy z `exc_info`)
6. **Fix list** (bugi do naprawy jako osobne PR-y — jeden PR per bug)

5. Konwencje raportu

- **CRIT** **CRITICAL** — bug produkcyjny, fix w tym audycie
- **HIGH** **HIGH** — poważny antypattern, fix zaraz po CRITICAL
- **MED** **MEDIUM** — do refaktora w osobnym cyklu
- **OK** **LOW** — cosmetic / nice-to-have

Każde znalezisko musi mieć:

- **Lokalizacja:** `plik.py:linia`
- **Opis problemu**
- **Skutek** (co się popsuje na prod)
- **Rekomendacja** (konkretna zmiana)
- **Estymata** (S / M / L)

Audyt techniczny — Allegro

Zakres: `zdrovena/common/allegro.py`, `zdrovena/api/routers/allegro_poller.py`, `zdrovena/api/routers/fakturownia_patcher.py`, sekcje Allegro w `zdrovena/api/routers/webhooks.py`.

Metodologia: kontraktowy audyt względem dokumentacji Allegro Developer + `ruff check --select ALL` (strict) + `pyright` + manual review.

Wynik pyright: OK 0 errors, 0 warnings (wszystkie 3 pliki Allegro). **Wynik ruff strict:** CRIT 15 znalezisk w poller + patcher, 11 znalezisk w `allegro.py`, 12 znalezisk w sekcji Allegro `webhooks.py`.

1. Kontraktowe niezgodności z API Allegro (CRITICAL)

Poniżej pełna tabela znalezionych rozbieżności między naszym kodem a rzeczywistym API. **Wszystkie 5 powodują 400 Bad Request / brakujące dane w produkcji.**

#	Severity	Plik : Linia	Endpoint	Nasz kod	Wg dokumentacji	Fixture
1	CRIT CRITICAL	<code>fakturownia_patcher.py:119</code>	GET /order/checkout-forms/{id}/invoices	<code>allegro_inv.get("number")</code>	<code>allegro_inv.get("invoiceNumber")</code>	<code>fixtures/allegro_get_invoices.json</code>
2	CRIT CRITICAL	<code>allegro.py:323-341</code> + <code>webhooks.py:455-480</code>	POST /shipping/drafts/{id}/create-commands	<code>weight.amount</code> , <code>dimensions.length.amount</code> , <code>unit: "KILOGRAM"</code>	<code>weight.value</code> , <code>flat length.value</code> (bez <code>dimensions</code> wrapper), <code>unit: "KILOGRAMS"</code> (plural), <code>type: "PACKAGE"</code> wymagane	<code>fixtures/allegro_create_commands_request.json</code>
3	CRIT CRITICAL	<code>allegro.py:323-341</code> + <code>webhooks.py:455-480</code>	POST /shipping/drafts/{id}/create-commands	Wysyłamy <code>orderId</code> (pole nie istnieje w API)	Wymagane bloki <code>sender</code> + <code>receiver</code> z pełnym adresem/telefonem/email	j.w.
4	CRIT CRITICAL	<code>allegro.py:328</code> + <code>webhooks.py:468</code>	POST /shipping/drafts/{id}/create-commands	<code>additionalServices: {"sendingAtPoint": "parcel_locker"} (dict)</code>	<code>additionalServices: ["sendingAtPoint"]</code> (Array of strings). Wartość <code>"parcel_locker"</code> nie występuje w API.	j.w.

#	Severity	Plik : Linia	Endpoint	Nasz kod	Wg dokumentacji	Fixture
5	CRIT CRITICAL	allegro.py:33 5 + webhooks.py:474	POST / shipping/ drafts/{id}/ create- commands	input.pickupPo intId	input.receiver .point	j.w.

Efekt sumaryczny: cały flow "Wysyłam z Allegro" (feature dostarczony w PR #72) jest **nieoperacyjny** — 4 z 5 bugów są w jednym endpointzie i każdy pojedynczo powoduje 400 Bad Request. Bug #1 (invoiceNumber) sprawia, że `fakturownia_patcher` nie potrafi zidentyfikować faktury do dopięcia kaucji — mockowane testy przechodzą, bo używają fikcyjnego pola `number`.

1.1 Bug #6 — dokumentowe endpointy DELETE nie istnieją

`webhooks.py:1-11` docstring reklamuje endpointy DELETE `/shipping/drafts/{id}/shipment` i `/shipping/drafts/{id}/dispatch`. Router ich nie definiuje. Decyzja: **dodać brakujące endpointy w finalnym PR** (potwierdzone przez usera).

1.2 Bug #7 — pending martwe endpointy

7 z 16 endpointów Allegro w `allegro.py` nie ma call site w projekcie:

- `get_order`, `get_shipments`, `get_delivery_services`, `get_delivery_proposal`,
`create_invoice_declaration`, `upload_invoice_file`, `get_ship_with_allegro_label`.

Rekomendacja: usunąć lub oznaczyć `# noqa: unused-public-api` z uzasadnieniem.

2. Endpointy — inwentaryzacja i status

2.1 Klient `AllegroClient` (16 metod, 10 użytkowane)

#	Method	Path	Idempotent	Retry safe	Użytkowane?
1	POST	<code>/auth/oauth/ token</code>	Tak (token refresh)	OK	OK (<code>_fetch_token</code>)
2	GET	<code>/order/checkout- forms?status=...</code>	Tak	OK	OK (<code>allegro_poller</code>)
3	GET	<code>/order/checkout- forms/{id}</code>	Tak	OK	CRIT martwy
4	PUT	<code>/order/checkout- forms/{id}/ fulfillment</code>	Tak	OK	OK (<code>mark_order_processed</code> , poller)
5	GET	<code>/order/checkout- forms/{id}/ invoices</code>	Tak	OK	OK (<code>fakturownia_patcher</code>)
6	POST	<code>/order/checkout- forms/{id}/ invoices</code>	X NIE	X (idempotent tylko przez check-then-post)	OK (<code>create_invoice_declaration</code> — martwe)

#	Method	Path	Idempotent	Retry safe	Użytkowane?
7	PUT	<code>/order/checkout-forms/{id}/invoices/{invoiceId}/file</code>	Tak (upload)	OK	CRIT martwy
8	GET	<code>/order/checkout-forms/{id}/shipments</code>	Tak	OK	CRIT martwy
9	POST	<code>/order/checkout-forms/{id}/shipments</code>	X	X	OK (tracking push, <code>webhooks.py:325</code>)
10	GET	<code>/sale/delivery-services</code>	Tak	OK	CRIT martwy
11	GET	<code>/shipping/carriers</code>	Tak	OK	OK (<code>webhooks.py:735</code>)
12	POST	<code>/shipping/drafts/{id}/delivery-proposals</code>	X	X	CRIT martwy
13	POST	<code>/shipping/drafts/{id}/create-commands</code> (SHIPMENT)	X	X	OK (<code>create_ship_with_allegro_shipment</code>) CRIT 5 błądów kontraktowych
14	POST	<code>/shipping/drafts/{id}/create-commands</code> (DISPATCH)	X	X	OK (<code>create_ship_with_allegro_dispatch</code>)
15	GET	<code>/shipping/drafts/{id}/commands/{cmdId}</code>	Tak	OK	OK (polling <code>AllegroCommandPending</code>)
16	GET	<code>/shipment-management/shipments/{id}/label</code>	Tak	OK	CRIT martwy

2.2 Router własny — 6 endpointów publicznych (`webhooks.py`)

#	Method	Path	Idempotent?	Idempotency-Key?	Uwagi
1	POST	<code>/allegro/poll-orders</code>	Semi (drafts)	X	manualny trigger; poller wewnętrznie dedupuje po <code>order.id</code>
2	POST	<code>/allegro/patch-invoices</code>	Semi	X	Fakturownia PATCH sam sprawdza istniejący settlement

#	Method	Path	Idempotent?	Idempotency-Key?	Uwagi
3	POST	<code>/allegro/orders/{id}/push-tracking</code>	X	X	wielokrotny call → wielokrotne shipmenty w Allegro
4	POST	<code>/allegro/drafts/{id}/create-shipment</code>	X	X	wielokrotny call → wielokrotne commands (koszt!)
5	POST	<code>/allegro/drafts/{id}/create-dispatch</code>	X	X	wielokrotny call → wielokrotne dispatch
6	GET	<code>/allegro/commands/{id}/{cmdId}</code>	Tak	OK	polling status

Brakujące DELETE:

- `DELETE /shipping/drafts/{id}/shipment` (cancel przed dispatch)
- `DELETE /shipping/drafts/{id}/dispatch` (cancel przed acceptance)

3. Antypatyczny — ruff strict (26 znalezisk w 4 plikach)

3.1 `allegro.py` (11 znalezisk)

Reguła	Ile	Linie	Severity	Opis
PLR2004	5	168, 173, 182, 191	MED medium	Magic values (400, 500, 204). Fix: wprowadzić <code>HTTP_BAD_REQUEST = 400</code> , <code>HTTP_NO_CONTENT = 204</code> itd.
S101	1	124	MED medium	<code>assert self._access_token is not None</code> — assert w kodzie prod. Fix: raise własny lub cast.
ERA001	1	293	OK low	Commented-out code (link do dokumentacji). Fix: przenieść do docstringa metody.
E501	1	293	OK low	Line too long (104 > 100). Fix po ERA001.
PLC0415	1	441	OK low	<code>import base64</code> w środku funkcji <code>get_ship_with_allegro_label</code> (i tak martwe). Fix: usunąć martwe.

3.2 `allegro_poller.py` (5 znalezisk — wszystkie o exception handling)

Reguła	Ile	Linie	Severity	Opis
BLE001	4	56, 67, 87, 100	CRIT high	<code>except Exception</code> — łapie WSZYSTKO włącznie z <code>KeyboardInterruption</code> semantycznie, gubi <code>CourierServerError</code> → <code>CourierTransientError</code> . Fix: <code>except (AllegroBusinessError, AllegroAuthError, CourierTransientError) as exc:</code>
TRY400	3	57, 68, 88	CRIT high	<code>logger.error</code> w bloku <code>except</code> bez traceback. Fix: <code>logger.exception(...)</code> (auto <code>exc_info=True</code>).

3.3 `fakturownia_patcher.py` (7 znalezisk)

Reguła	Ile	Linie	Severity	Opis
BLE001	3	100, 113, 132	CRIT high	<code>except Exception</code> — jak w pollerze. Fix: typowane wyjątki + fallback <code>Exception</code> z re-raise.
TRY400	3	101, 114, 133	CRIT high	j.w. <code>logger.exception(...)</code> obowiązkowo w bloku except.
RUF002	2	10, 77	OK low	Ambiguous <code>x</code> (MULTIPLICATION SIGN) w docstringu. Fix: <code>x</code> lub <code>*</code> .

3.4 Sekcja Allegro `webhooks.py` (12 znalezisk)

Reguła	Ile	Linie	Severity	Opis
BLE001	5	100, 187, 321, 528, 732	CRIT high	Endpointy publiczne — łapia wszystko i zwracają 500. Powinny łapać typowane wyjątki + zwracać 4xx/5xx per HTTP semantyka.
TRY400	4	188, 870, 935, 1040	CRIT high	<code>logger.exception</code> w except.

Reguła	Ile	Linie	Severity	Opis
TRY003	2	446, 451	OK low	Długie wiadomości w <code>HTTPException(detail=...)</code> . OK dla routera FastAPI (użytkownik potrzebuje info) — można whitelistować .
TRY301	1	1036	OK low	<code>raise</code> z outer <code>try</code> . OK jeśli intencjonalne.

4. Idempotency

4.1 Idempotent (safe do retry)

- `list_orders`, `list_order_invoices`, `get_command_status`, `get_shipping_carriers` — GET-y, safe.
- `mark_order_processed` — PUT z tym samym payloadem. Allegro akceptuje, ale **każdy call = koszt API**. Fix: `if form["fulfillment"]["status"] == "PROCESSING": skip`.

4.2 NIE-idempotent (wymaga guard)

Endpoint	Ryzyko	Fix
<code>push_shipment_tracking</code> (POST shipments)	Duplikat = wielokrotne shipments w Allegro	Dodać kolumnę <code>allegro_tracking_pushed_at</code> w <code>shipping_store</code>
<code>create_ship_with_allegro_shipment</code>	Duplikat = wielokrotne commands (opłata za każdy!)	Sprawdzić <code>draft.status IN ('SHIPMENT_CREATED', 'DISPATCH_ACCEPTED')</code>
<code>create_ship_with_allegro_dispatch</code>	j.w.	j.w.
POST <code>/allegro/drafts/{id}/create-shipment</code>	j.w., publiczny endpoint	Wymagać <code>Idempotency-Key</code> header w opts, dedup w Redis (5min TTL)

4.3 Rekomendacja: `Idempotency-Key` na wszystkich POST-ach

Wszystkie publiczne endpointy `/allegro/...` typu POST powinny obowiązkowo przyjmować header `Idempotency-Key: <uuid>` i dedupować w Redis (30 min TTL). Cache klucza + odpowiedzi (nie samego "widziałem"). Duplikat → zwróć cached response.

5. Exception mapping — audyt

5.1 Hierarchia w `shipping_exceptions.py` (istniejąca, dobra baza)

```

ZdrovenaError
├── ZdrovenaShippingError
│   ├── ShopifyPayloadError → 400 (walidacja)
│   └── CourierAuthError → 502 (auth do dostawcy)

```

```

|   |— AllegroAuthError
|   |— InPostAuthError
|   |— ApaczkaSignatureError
|   |— ApaczkaInsufficientBalanceError
|— CourierBusinessError → 502 (4xx z API dostawcy)
|   |— AllegroBusinessError → AllegroCommandPending
|— CourierTransientError → 502 (retry-able)
|   |— CourierTimeoutError
|   |— CourierConnectionError
|   |— CourierServerError
|— CancellationError → 409

```

Ocena: hierarchia Allegro (`AllegroAuthError`, `AllegroBusinessError`, `AllegroCommandPending`, `CourierServerError` dla 5xx) jest **poprawna i kompletna**. `_request()` w `allegro.py:158-175` prawidłowo mapuje status code → wyjątek.

5.2 Problem — łapanie `except Exception`

Przykład (`allegro_poller.py:54-59`):

```

try:
    forms = client.list_orders(status=status)
except Exception as exc:          # BLE001 <span class="badge-crit">CRIT</span>
    logger.error("Allegro list_orders failed: %s", exc) # TRY400 <span class="badge-crit">CRIT</span>
    stats["errors"] += 1
    return stats

```

Co się gubi:

1. `CourierTransientError` powinno być retry-owane (backoff), nie połącznięte.
2. `CourierAuthError` powinno być **alertem** (Slack/PagerDuty) — token się zepsuł.
3. Traceback jest tracony (brak `exc_info=True`).

Fix (przykład dla poller):

```

try:
    forms = client.list_orders(status=status)
except CourierTransientError:
    logger.exception("Allegro transient error – will retry next cycle")
    stats["errors"] += 1
    stats["retryable_errors"] += 1
    return stats
except (AllegroAuthError, CourierAuthError):
    logger.exception("Allegro AUTH FAILURE – check token/creds")
    stats["errors"] += 1
    stats["auth_errors"] += 1
    raise # niech alertuje wyżej
except AllegroBusinessError:
    logger.exception("Allegro business error – likely bad request")
    stats["errors"] += 1
    return stats

```


5.3 `InPostError` / `ApaczkaError` — poza hierarchią (heads-up dla InPost/Apaczka audytów)

`inpost.py:107` i `apaczka.py:28` definiują `class InPostError(Exception)` i `class ApaczkaError(Exception)` NIE dziedzicząc z `ZdrovenaShippingError`. Do refaktoru w audytach InPost/Apaczka.

6. Logging

6.1 Standard obowiązujący (do włączenia)

- Każdy `except` musi używać `logger.exception(...)` (nie `.error()` z `%s` na exc).
- Log message w formacie `<Klasa/Serwis> <akcja>: <konkret>` — np. `"Allegro list_orders failed"` (nie `"list_orders failed"`).
- Log level:
 - `INFO` — happy path (POST created, PUT succeeded).
 - `WARNING` — retry-able (transient, auth expired, empty result).
 - `ERROR` (przez `.exception()`) — nieoczekiwane, breaking.
- **NIE** logować payloadu z access tokenem, credentials, PII (email/tel klienta) w plain-text.

6.2 Obecny stan w plikach Allegro

Plik	<code>.error()</code> z exc	<code>.exception()</code>	Standard?
<code>allegro.py</code>	0	0	brak (używa raise)
<code>allegro_poller.py</code>	3 (linie 57,68,88)	0	CRIT do naprawy
<code>fakturownia_patcher.py</code>	3 (linie 101,114,133)	0	CRIT do naprawy
<code>webhooks.py</code> (Allegro sekcja)	4 (linie 188,870,935,1040)	0	CRIT do naprawy

7. Lista fixów do finalnego PR (Allegro)

Blok A — Kontrakty API (CRITICAL, 5 bugów blokujących feature Wysyłam z Allegro):

- ☐ **F1:** `fakturownia_patcher.py:119` — zmień `"number"` → `"invoiceNumber"`. Zaktualizuj 15 testów w `tests/test_fakturownia_patcher.py`.
- ☐ **F2:** `allegro.py:323-341` — przebuduj `packages` structure (`weight.value`, flat `length.value`, `unit: "KILOGRAMS"`, `type: "PACKAGE"`).
- ☐ **F3:** `allegro.py:323-341` + `webhooks.py:455-480` — dodać `sender` / `receiver` blocks (dane z `shipping_store` draftu, decyzja usera).
- ☐ **F4:** `allegro.py:328` + `webhooks.py:468` — `additionalServices` z dict → Array of strings.
- ☐

F5: `allegro.py:335` + `webhooks.py:474` — `pickupPointId` top-level → `input.receiver.point`.

Blok B — Endpointy DELETE (feature completion):

- ☐ F6: dodać `DELETE /allegro/drafts/{id}/shipment` → `allegro.py.cancel_ship_with_allegro_shipment(draft_id)` → `DELETE /shipping/drafts/{id}/shipment`.
- ☐ F7: dodać `DELETE /allegro/drafts/{id}/dispatch` → analogicznie.

Blok C — Exception handling (Allegro-only, 12 miejsc):

- ☐ F8: `allegro_poller.py:56,67,87,100` — zastąp `except Exception` typowanym łapaniem.
- ☐ F9: `fakturownia_patcher.py:100,113,132` — j.w.
- ☐ F10: `webhooks.py:100,187,321,528,732` (sekcja Allegro) — j.w.
- ☐ F11: zamień 10× `logger.error(..., exc)` na `logger.exception(...)`.

Blok D — Idempotency:

- ☐ F12: dodać `allegro_tracking_pushed_at` do `shipping_store`, w `push_shipment_tracking` skipnij jeśli set.
- ☐ F13: `mark_order_processed` — skip jeśli `fulfillment.status == "PROCESSING"`.
- ☐ F14: publiczne POST-y `/allegro/...` → wymóg `Idempotency-Key` header + Redis dedup.

Blok E — Cleanup:

- ☐ F15: usunąć 6 martwych metod (`get_order`, `get_shipments`, `get_delivery_services`, `get_delivery_proposal`, `create_invoice_declaration`, `upload_invoice_file`, `get_ship_with_allegro_label`).
- ☐ F16: `allegro.py:293` — commented-out link do docs → docstring.
- ☐ F17: `allegro.py:168,173,182,191` — magic values → stałe `HTTP_*`.
- ☐ F18: `fakturownia_patcher.py:10,77` — `x` → `x`.
- ☐ F19: docstring `webhooks.py:1-11` — usuń fałszywą wzmiankę o DELETE albo poczekać po F6/F7.

Estymacja rozmiaru fixu: ~450 linii kodu produkcyjnego + ~300 linii testów. **1 PR** (decyzja usera).

Załączniki

- [fixtures/allegro_get_invoices.json](#) — realny kontrakt `GET /order/checkout-forms/{id}/invoices` (pole `invoiceNumber`)
- [fixtures/allegro_post_shipment_request.json](#) — realny kontrakt `POST shipments`
- [fixtures/allegro_create_commands_request.json](#) — poprawny schemat `POST create-commands`

Źródła:

- [Allegro Developer — Dokumentacja](#)
- [Wysłałam z Allegro — Tutorial](#)
- Content-Type: `application/vnd.allegro.public.v1+json`

Audyt techniczny — Fakturownia

Zakres: `zdrovena/common/fakturownia.py` (261 linii, klient REST).

Metodologia: kontraktowy audyt względem dokumentacji Fakturownia API + KSeF FA(3) field mapping + `ruff strict` + `pyright`.

Wynik pyright: OK 0 errors. **Wynik ruff strict:** MED 14 znalezisk (głównie kosmetyczne: magic values HTTP 200/300/500/600, EM101/EM102/TRY003 dla stringów w exceptions).

Ogólna ocena: Bardzo czysty klient. Kontrakt endpointów zgodny z publiczną dokumentacją. Główne ryzyko: `settlement_positions` field nie jest publicznie udokumentowane (widoczne tylko w KSeF FA(3) mapping, bez schematu wierszy).

1. Kontraktowe niezgodności z API Fakturownia

Zero bugów blokujących. Endpointy, metody i parametry zgodne z dokumentacją.

1.1 Ryzyko krytyczne: `settlement_positions` — undocumented

Aspekt	Nasz kod	Dokumentacja	Ryzyko
Nazwa pola	<code>"settlement_positions"</code> (<code>fakturownia.py:203,231</code>)	Wymienione w KSeF FA(3) mapping (Rozliczenie → obciążenia/odliczenia). Brak szczegółu struktury wiersza.	MED medium
<code>kind</code> values	<code>"charge"</code> , <code>"deduction"</code> (<code>fakturownia.py:47-48</code>)	Brak publicznej dokumentacji tych wartości.	CRIT high
Format kwoty	String <code>"0.50"</code> (via <code>_normalize_amount_pln</code>)	Brak publicznej dokumentacji.	MED medium
Klucz opisu	<code>"description"</code>	Brak publicznej dokumentacji.	MED medium
PUT semantics	Przekazujemy pełną listę (z istniejącymi rowami + nowym)	Nieudokumentowana, potwierdzona empirycznie	MED medium
<code>id</code> zachowanie istniejących	Kod NIE kopiuje <code>id</code> z istniejących wierszy (<code>fakturownia.py:222</code>) — używa <code>list(existing_rows)</code> (referencje, nie kopie!)	Rails PUT semantics zwykle wymagają <code>id</code> na kept rows	MED medium

Bug #F2-1 (MED medium): `fakturownia.py:222` — `merged: list[dict[str, Any]] = list(existing_rows)` tworzy **shallow copy** listy, ale dict-y w środku są **shared reference**. Jeśli między `get_invoice` a `update_invoice` inny worker zmienił `settlement_positions` w bazie, to nasz PUT nadpisze ich zmiany (last-write-wins). Fix: deep copy + weryfikacja czy istniejące `id` są kompletne.

Bug #F2-2 (MED medium): docstring komentarza w `fakturownia.py:221` mówi *"Preserve existing rows verbatim (Rails PUT semantics require `id` on kept rows)"* — ale kod NIE zapewnia obecności `id`. Fix: filter/assert że każdy row z `existing_rows` ma `id` przed dodaniem do merged list.

Rekomendacja: napisać kontrakt-test integracyjny (opt-in, przeciwko sandboxowi Fakturownia) który weryfikuje semantykę PUT z `settlement_positions`. Alternatywnie — zamówić potwierdzenie od Fakturownia support (email: pomoc@fakturownia.pl) że `charge` / `deduction` są supported values dla `kind` w API i że schema jest stabilna.

1.2 Uzupełnienia dobrej praktyki

- `_request` (`fakturownia.py:89-113`) **poprawnie** obsługuje `requests.Timeout` → `CourierTimeoutError`, `requests.ConnectionError` → `CourierConnectionError`. +
- `_parse_response` (`fakturownia.py:115-136`) **poprawnie** mapuje status codes na hierarchię wyjątków:
 - `401/403` → `FakturowniaAuthError` OK
 - `5xx` → `FakturowniaServerError` OK
 - `4xx (inne)` → `FakturowniaBusinessError` OK
 - `2xx` → JSON body OK
- `add_settlement_position` (`fakturownia.py:174-231`) **poprawnie** implementuje idempotency guard (linie 210-219): sprawdza duplikat description po `casefold()` na `existing_rows` przed PUT. +
- `has_settlement_with_description` (`fakturownia.py:233-245`) — statyczna helper dla wołającego (patcher), umożliwia early-skip bez wykonania GET. +

2. Endpointy — inwentaryzacja

#	Method	Path	Idempotent	Retry safe	Użytkowane?
1	GET	<code>/invoices/{id}.json</code>	Tak	OK	OK (<code>get_invoice</code>)
2	GET	<code>/invoices.json?...<div data-bbox="368 614 484 658" data-label="Text">...</div></code>	Tak	OK	OK (<code>list_invoices</code>) — filtry: period, oid, number, include_positions)
3	PUT	<code>/invoices/{id}.json</code>	Tak (PUT semantics)	OK	OK (<code>update_invoice</code>)
4	Composite	<code>add_settlement_position</code> (GET→PUT)	Tak (double-check idempotency)	OK	OK (<code>fakturownia_patcher.py</code>)

Zero martwych endpointów. Klient minimalistyczny, tylko to co używane.

Brakuje: `POST /invoices.json` (create) — używane w kodzie? Nie widziałem call site. **OK, celowo poza scope.**

3. Antypaterny — ruff strict (14 znalezisk)

3.1 `fakturownia.py`

Reguła	Ile	Linie	Severity	Opis
PLR2004	5	118, 119, 133 (x2), 118	low	Magic values (200, 300, 204, 500, 600). Fix: stałe HTTP_OK, HTTP_MULTIPLE_CHOICES itd. Do wspólnego cross-cutting fixu.
TRY003	4	79, 193, 197, 200	low	Długie messages w <code>raise ValueError(...)</code> . Fix: własne exception classes ALBO whitelist reguły.
EM101/EM102	4	80, 193, 197, 200	low	Powiązane z TRY003 — string literal directly in raise. Fix jak wyżej.
PYI041	1	64	low	<code>`int</code>

Ocena: wszystkie znaleziska są **kosmetyczne / stylowe**. Zero antypaternów bezpieczeństwa, zero antypaternów wydajności, zero blind-exception. To wzorcowy plik do naśladowania w reszcie kodu.

3.2 `fakturownia_patcher.py` (endpointy)

Zobacz Raport Allegro §3.3 (bo patcher jest wspólny — audytowany razem z Allegro). Kluczowe znaleziska:

- **BLE001** (3× — linie 100, 113, 132) — `except Exception`. Fix wspólny z pollerem.
- **TRY400** (3× — linie 101, 114, 133) — `.error` → `.exception`.
- **RUF002** (2× — linie 10, 77) — `MULTIPLICATION SIGN` w docstringu → `x`.

4. Idempotency

4.1 Klient (`fakturownia.py`)

- `get_invoice`, `list_invoices`, `update_invoice` — GET/PUT semantics, natywnie idempotent.
- `add_settlement_position` — **explicit double-check idempotency**:
 1. Wołający robi `has_settlement_with_description(current_invoice, desc)` (early skip).
 2. Klient wewnątrz `add_settlement_position` PONOWNIE robi GET + check (race protection).
Wzorzec **prawidłowy**.

4.2 Rekomendacja

Dodać w `FakturowniaClient.__init__` opcjonalny param `idempotency_key: str | None = None` — jeśli set, przekazywać jako header `Idempotency-Key` (Fakturownia obecnie nie honoruje, ale gotowość na przyszłość + audit trail w naszych logach).

5. Exception mapping — audyt

5.1 Klient (`fakturownia.py`) — WZORZEC POZYTYWNY

Klient **poprawnie i kompletnie** mapuje wszystkie 3 rodzaje błędów na typowane wyjątki z

`shipping_exceptions.py`:

```
requests.Timeout          → CourierTimeoutError(courier="fakturownia") # transient, retry
requests.ConnectionError → CourierConnectionError(courier=..., detail=...) # transient
HTTP 401/403              → FakturowniaAuthError(detail=...) # auth failure, alert
HTTP 5xx                  → FakturowniaServerError(status=...) # transient
HTTP 4xx (inne)           → FakturowniaBusinessError(detail=..., action=...) # bad request
HTTP 2xx                  → JSON body (Any)
```

To wzorzec do skopiowania do klientów InPost i Apaczka (które nie mają hierarchii).

5.2 Patcher (`fakturownia_patcher.py`) — problematyczny

- **BLE001** w 3 miejscach — patcher powinien łączyć `FakturowniaAuthError`, `FakturowniaBusinessError`, `CourierServerError` osobno i mapować:
 - `FakturowniaAuthError` → alert + `raise`
 - `FakturowniaBusinessError` → log + `continue` (per invoice, nie blokuje batcha)
 - `CourierServerError` / `CourierTransientError` → log + retry counter, continue

Fix wspólny z Allegro (Blok C w Raporcie Allegro).

6. Logging

6.1 Klient — cichy

`fakturownia.py` używa `log = logging.getLogger(__name__)` i loguje TYLKO 1 rzecz:

`log.info("add_settlement_position: invoice %s already has row %r — skipping PUT", ...)` (linia 214).

Rekomendacja: dodać INFO log na successful update_invoice (audit trail dla PIT/KSeF), WARNING na retry (`FakturowniaServerError`).

6.2 Patcher — do naprawy

3× `.error` z exc → 3× `.exception()` (Fix F11 w Raporcie Allegro).

Rekomendacja dodatkowa: `fakturownia_patcher.py:126-141` (patchowanie 1 faktury) — brakuje logu INFO na happy path. Powinno być:

```
log.info("Patched invoice %s (order %s): added kaucja %s PLN", invoice_number, order_id,
kaucja_amount).
```


7. Lista fixów do finalnego PR (Fakturownia)

Blok A — Undocumented API contract (medium):

- ☐ **FA1:** Dodać integration test (opt-in, `@pytest.mark.integration`) który tworzy testową fakturę w sandboxie Fakturownia, PATCH-uje `settlement_positions` z `kind=charge`, waliduje że GET zwraca zapisany row. Docelowo run raz na tydzień w CI (nightly).
- ☐ **FA2:** `fakturownia.py:222` — deep copy `existing_rows` przed PATCH-em (zapobiega mutacji przez shared ref).
- ☐ **FA3:** `fakturownia.py:222` — assert że każdy row w `existing_rows` ma `id` (fail-fast jeśli Fakturownia zmieni schema).

Blok B — Cosmetics (low, do cross-cutting):

- ☐ **FA4:** `fakturownia.py:64` — `int | float` → `float`.
- ☐ **FA5:** `fakturownia.py:118,133` — magic HTTP status codes → stałe (razem z Allegro Fix F17).
- ☐ **FA6:** `fakturownia.py:79-80,193,197,200` — TRY003/EM101/EM102 → własne exception classes ALBO whitelist tych reguł w `pyproject.toml` (rekomenduję whitelist — `ValueError` z tekstem jest OK, to Pythonic).

Blok C — Exception & logging (wspólne z Allegro):

- ☐ Patcher: fixy F9, F11 z Raportu Allegro.
- ☐ Klient: dodać `log.info` na `update_invoice` (audit trail).

Estymacja: ~50 linii kodu, 1 integration test file (~80 linii). **W tym samym gigantycznym PR co Allegro.**

Załączniki

Źródła:

- [Fakturownia API — Publiczna dokumentacja](#)
- [Fakturownia KSeF FA\(3\) field mapping \(2026-06-11\)](#)
- [Fakturownia GitHub API examples](#)

Audyt techniczny — Shopify

Zakres: `zdrowena/api/routers/webhooks.py` — endpoint `POST /webhooks/shopify/order-created` (linie 737-800) + helpery (`_verify_shopify_hmac`, `_is_duplicate_webhook`).

Metodologia: kontraktowy audyt względem [Shopify webhook docs \(Verify webhook deliveries\)](#) + `ruff strict` + manual review.

Wynik pyright: `OK` 0 errors. **Wynik ruff strict:** 12+ znalezisk w `webhooks.py` (całość pliku, tylko podzbiór dot. Shopify).

1. Kontraktowe niezgodności z Shopify Webhook API

1.1 **CRIT** CRITICAL — `X-Shopify-Webhook-Id` czytany ale nie persystowany

Bug #S1 (CRITICAL) — `webhooks.py:752` czyta `X-Shopify-Webhook-Id`, ale `_is_duplicate_webhook` (linia 719-734) go **NIE używa** do deduplication. Zamiast tego porównuje `shopify_order_id` z listy draftów.

Dokumentacja Shopify wymaga ([source](#)):

"If your processing isn't idempotent, use the `X-Shopify-Webhook-Id` header to detect and skip duplicates."

1. Extract `X-Shopify-Webhook-Id` from headers.
2. Check your persistent store for that ID.
3. If it exists, skip. If new, process AND save the ID.

Problem produkcyjny: order może mieć wiele webhooków (order-created + order-updated + order-fulfilled). Sprawdzanie tylko `order_id` **zablokuje** legalne re-webhooki dla update'ów.

Fix (F-S1): dodać nową tabelę/entity `shopify_webhook_delivery_id` (tylko `webhook_id` + `processed_at`). W `_is_duplicate_webhook`:

```
def _is_duplicate_webhook(webhook_id: str, shipping_store: ShippingStore) -> bool:
    if not webhook_id:
        return False
    return shipping_store.webhook_delivery_exists(webhook_id)
```

Po sukcesie procesowania: `shipping_store.record_webhook_delivery(webhook_id)`.

1.2 **CRIT** CRITICAL — `ALLOW_UNSIGNED_SHOPIFY_WEBHOOKS` bypass

Bug #S2 (CRITICAL) — `webhooks.py:766-782`. Jeśli `shopify_webhook_secret` nie jest ustawiony i env var `ALLOW_UNSIGNED_SHOPIFY_WEBHOOKS=1`, endpoint **akceptuje niesignowane webhooks**. Shopify docs jednoznacznie mówią:

"Always verify HMAC before trusting payload contents."

Ryzyko: atakujący z URL-em endpointu może wstrzykiwać dowolne "zamówienia" → utworzą się drafty → zostaną wysłane paczki na obcy adres.

Fix (F-S2):

- Usunąć `ALLOW_UNSIGNED_SHOPIFY_WEBHOOKS` całkowicie (**preferowane**).
- Alternatywa jeśli potrzebne do testów: zostawić flagę ALE tylko w `DEV` / `STAGING` env, w `PRODUCTION` forced-fail. Można egzekwować: `if os.getenv("APP_ENV") == "prod" and allow_unsigned: raise RuntimeError`.

1.3 **MED** MEDIUM — Race condition w idempotency check

Bug #S3 (MEDIUM) — Duplicate check odbywa się przed

`background_tasks.add_task(_create_draft, ...)`. Dwa równoczesne webhooki (retry) mogą OBIEMinąć check zanim pierwszy zapisze draft, tworząc 2 drafty.

Fix (F-S3): przenieść dedup check do `_create_draft` na poziomie storage (INSERT ... IF NOT EXISTS na kluczu `shopify_order_id`), lub użyć locka/leasingu (Azure Table blob lease).

1.4 **MED** MEDIUM — Silent exception w `_is_duplicate_webhook`

Bug #S4 (MEDIUM) — `webhooks.py:732-733`:

```
except Exception:
    pass
```

Jeśli `shipping_store.list_drafts()` rzuci, uznajemy że NIE jest duplikatem → tworzymy draft. Może zadziałać nawet gdy storage jest niedostępny → **duplikat draftu przy każdym retry**.

Fix (F-S4): log exception (nie połącz) + fail-safe: `raise HTTPException(503)` (Shopify wtedy retry-uje).

1.5 **MED** MEDIUM — O(N) na każdy webhook

Bug #S5 (MEDIUM) — `_is_duplicate_webhook` iteruje po **wszystkich draftach**

(`shipping_store.list_drafts()`). Przy 10 000 draftów: każdy webhook = 10k rows scan.

Fix (F-S5): dodać indeks/query `get_draft_by_shopify_order_id(order_id)` w `ShippingStore` (Azure Table row key = order_id ⇒ O(1) lookup). Alternatywnie odejść od tego dedupu i użyć webhook_id (Fix F-S1).

1.6 **MED** MEDIUM — 5-sekundowy timeout Shopify

Kontekst: Shopify wymaga odpowiedzi w 5 sekund ([source](#)):

"Shopify has a one-second connection timeout and a five-second timeout for the entire request."

Nasz kod: używa `background_tasks.add_task(_create_draft, ...)` (linia 798), więc endpoint zwraca 200 przed wykonaniem heavy work. **OK Dobra praktyka.**

ALE: `_is_duplicate_webhook` na 10k draftów może samo w sobie zająć >5s (Azure Table full scan). Fix F-S5 to rozwiązuje.

1.7 OK LOW — Brak walidacji `X-Shopify-Topic`, `X-Shopify-Shop-Domain`

Bug #S6 (LOW) — endpoint akceptuje HMAC-poprawny payload z DOWOLNEGO tematu / shop. Sensowne jest weryfikować:

- `X-Shopify-Topic == "orders/create"` (lub `orders/paid`)
- `X-Shopify-Shop-Domain == "zdrowena.myshopify.com"` (whitelist)

Bez tego: jeśli Shopify (przez pomyłkę konfiguracji) wyśle webhook z topic "products/create" — wciąż przejdzie HMAC (bo sekret ten sam) i utworzy draft na base ProductJSON → crash.

Fix (F-S6): dodać whitelist headers po HMAC verify.

2. Endpoint — inwentaryzacja

#	Method	Path	Idempotent?	HMAC?	Retry-friendly?
1	POST	<code>/webhooks/shopify/order-created</code>	Częściowo (order_id-based)	OK	OK zwraca 200 przez background task

Zero endpointów wychodzących do Shopify API — jesteśmy tylko odbiorcą webhooków (no outbound calls). To znacznie upraszcza audyt.

Skala: Shopify wysyła webhook per order. Retry policy: 8× w ciągu 4h przy błędzie/timeoucie. Po 8 consecutive failures subscription się kasuje.

3. Antypatery — ruff strict (Shopify sekcja)

W obrębie sekcji Shopify (webhooks.py:49-57, 719-800):

Reguła	Ile	Linie	Severity	Opis
BLE001	1	732	CRIT high	<code>except Exception: pass</code> w <code>_is_duplicate_webhook</code> → Bug #S4 (silent fail).
TRY400	1	870	CRIT high	<code>logger.error(..., exc)</code> w <code>execute_draft</code> (nie tylko Shopify, ale w tym samym pliku).
PLC0415	wiele	wiele	OK low	Imports w środku funkcji (np. <code>import os</code> w <code>_get_allegro_client</code> , <code>import base64</code>).

Reguła	Ile	Linie	Severity	Opis
FAST002	wiele	wiele	OK low	FastAPI: use <code>Annotated[X, Depends(...)]</code> style zamiast <code>x: X = Depends(...)</code> . Częściowo już zrobione.
ARG001	9	różne	OK low	Unused function argument (najczęściej <code>principal</code> — jest tylko dla auth guard). Whitelistować.

Ocena: sekcja Shopify sama w sobie NIE ma poważnych antypatrnów kodowych — bugi są kontraktowe (§1).

4. Idempotency

4.1 Obecny stan

Poziom	Mechanizm	Status
HTTP endpoint	Zwraca 200 zawsze (przez <code>background_task</code>)	OK
Idempotency key	Sprawdza <code>shopify_order_id</code> w liście draftów	MED O(N), nie używa <code>X-Shopify-Webhook-Id</code>
Race protection	Brak (fire-and-forget background task)	CRIT duplikaty możliwe
Success acknowledgment	Zwraca <code>{"status": "accepted"}</code> przed dokończeniem work	OK dobra praktyka dla 5s timeout

4.2 Zalecany model docelowy (po fixach)

```
POST /webhooks/shopify/order-created:
    raw_body = await request.body()
    webhook_id = headers["X-Shopify-Webhook-Id"]
    topic = headers["X-Shopify-Topic"]
    shop = headers["X-Shopify-Shop-Domain"]

    # 1. Verify HMAC (fail 401 if bad)
    # 2. Verify topic + shop in whitelist (fail 400 if bad)
    # 3. Check webhook_id in dedup table (return 200 if seen)
    # 4. INSERT webhook_id in dedup table (with race protection – UPSERT NOT EXISTS)
    # 5. If insert conflict: return 200 (duplicate)
    # 6. Otherwise: background_tasks.add_task(_create_draft, ...)
    # 7. Return 200
```

5. Exception mapping — audyt

5.1 Sekcja Shopify

Miejsce	Wyjątek	Response HTTP	Ocena
Missing HMAC header	<code>HTTPException(401)</code>	401 Unauthorized	OK
Invalid HMAC signature	<code>HTTPException(401)</code>	401 Unauthorized	OK
Missing webhook secret (bypass)	<code>HTTPException(503)</code> OR skip validation	503 lub skip	CRIT bypass to bug (Fix F-S2)
Invalid JSON body	<code>HTTPException(400)</code>	400 Bad Request	OK
Missing shipping_lines	Return <code>{"status": "skipped"}</code>	200	OK
Duplicate order	Return <code>{"status": "duplicate"}</code>	200	OK
<code>list_drafts</code> fails	<code>except Exception: pass</code>	Kontynuuje jakby nie było duplikatu	CRIT Fix F-S4
Background <code>_create_draft</code> fails	??? (poza scope endpointu)	Zawsze 200	MED potrzebna obserwowalność

5.2 Rekomendacja: obserwowalność background_task

`_create_draft` jest fire-and-forget. Jeśli rzuci, użytkownik dostaje 200 ale draft się nie utworzy. **Trzeba** dodać:

- Metric counter `shopify_webhook_processed{status="success|error"}` (Prometheus/App Insights).
- Alert w monitoring: `rate(shopify_webhook_processed{status="error"}[5m]) > 0`.

6. Logging

6.1 Obecny stan

Endpoint loguje:

- Missing HMAC → `logger.warning`
- HMAC mismatch → `logger.warning`
- Unsigned bypass → `logger.warning`
- Empty shipping_lines → `logger.warning`
- Duplicate → `logger.info` (via `_is_duplicate_webhook`)
- Draft queued → `logger.info`

Ocena: dobrze pokryte. Rekomendacja: dodać w każdym logu `X-Shopify-Webhook-Id` + `X-Shopify-Topic` + `X-Shopify-Shop-Domain` (correlation ID).

6.2 Alarmy do dodania

- **HIGH:** `HMAC mismatch` >0 w 5 min → **security alert** (ktoś próbuje forge webhooków).

- **MEDIUM:** `webhook_secret_not_configured` w prod → alert do inżyniera on-call.
- **LOW:** `Empty_shipping_lines` >5% webhooków → alert do Ops (bad Shopify config).

7. Lista fixów do finalnego PR (Shopify)

Blok A — Security (CRITICAL, blok w PR):

- ☐ **F-S1:** Nowa tabela `webhook_deliveries(webhook_id PK, order_id, topic, received_at)`. Refactor `_is_duplicate_webhook(webhook_id, ...)` — dedup po X-Shopify-Webhook-Id.
- ☐ **F-S2:** Usunąć `ALLOW_UNSIGNED_SHOPIFY_WEBHOOKS` z prod. W DEV/STAGING: OK, w PROD forced 503.
- ☐ **F-S6:** Whitelist `X-Shopify-Topic` (`orders/create`) i `X-Shopify-Shop-Domain` (zdrowena.myshopify.com).

Blok B — Race & performance (MEDIUM):

- ☐ **F-S3:** Storage-level INSERT ... IF NOT EXISTS w `_create_draft` (Azure Table `if_none_match=""`).
- ☐ **F-S4:** `_is_duplicate_webhook` — usuń `except Exception: pass`, log `.exception`, fail-safe raise 503.
- ☐ **F-S5:** `ShippingStore.get_draft_by_shopify_order_id(order_id)` — indeksowany lookup, nie full scan.

Blok C — Observability (nice-to-have):

- ☐ **F-S7:** Metric `shopify_webhook_processed{status}` + Prometheus/App Insights.
- ☐ **F-S8:** Correlation ID (`X-Shopify-Webhook-Id`) w każdym logu.

Estymacja: ~150 linii kodu produkcyjnego + ~200 linii testów (nowy `test_shopify_webhook_dedup.py`).

Załączniki

Źródła:

- [Shopify — Verify webhook deliveries](#)
- [Shopify — Webhook headers reference](#)

Audyt techniczny — InPost (ShipX)

Zakres: `zdrovena/common/inpost.py` (295 linii) — klient ShipX API + kod webhooks.py w sekcji InPost.

Metodologia: kontraktowy audyt względem [InPost ShipX 1.9.0 Shipment docs](#) + `ruff strict` + `pyright` + manual review.

Wynik pyright: OK 0 errors. **Wynik ruff strict:** MED 14 znalezisk (głównie RUF003 dla `<span class="badge-warn">?</span><span class="badge-crit">X</span><span class="badge-ok">OK</span>` w komentarzach, TRY003/EM102 dla `InPostError` raise'ów).

Ogólna ocena: kontrakt endpointów **zgodny** z ShipX API. Kluczowe problemy: `InPostError(Exception)` **poza hierarchią** `ZdrovenaShippingError` → utrudnia unified handling wyjątków w warstwie routera.

1. Kontraktowe niezgodności z InPost ShipX API

1.1 OK Zgodne z dokumentacją

Aspekt	Nasz kod	Dokumentacja	Status
Base URL	<code>https://api-shipx-pl.easypack24.net</code>	api-shipx-pl.easypack24.net (potwierdzone URL-em)	OK
Auth	<code>Authorization: Bearer <token></code>	Bearer token	OK
POST <code>/v1/organizations/{id}/shipments</code>	<code>receiver.first_name/last_name/email/phone</code> + <code>receiver.address.{street,building_number,city,post_code,country_code}</code> + <code>sender</code> + <code>parcels[]</code> + <code>service</code>	Peer object: name, company_name, first_name, last_name, email, phone, address. Parcel: dimensions {length,width,height,unit=mm} + weight {amount,unit=kg}	OK
<code>parcels</code> w courier	<code>dimensions.unit="mm"</code> , <code>weight.unit="kg"</code>	<code>unit: "mm"</code> (only), <code>unit: "kg"</code> (only)	OK
<code>parcels</code> w paczkomat	<code>[{"template": "large"}]</code>	<code>template: String</code> predefined	OK
<code>service</code> values	<code>inpost_locker_standard</code> , <code>inpost_courier_standard</code>	Zgodne z dokumentacją	OK
DELETE <code>/v1/shipments/{id}</code>	Kod wywołuje	Zgodne (klient obsługuje <code>422 → already dispatched</code>)	OK
DELETE <code>/v1/organizations/{id}/dispatch_orders/{id}</code>	Kod wywołuje	Zgodne (klient obsługuje <code>422 → already accepted</code>)	OK

Zero bugów kontraktowych w klienic InPost. +

1.2 MED MEDIUM — `custom_attributes.sending_method="dispatch_order"`

Bug #I1 (MEDIUM) — `inpost.py:143-145,194`. Wysyłamy `custom_attributes: {sending_method: "dispatch_order"}`. Dokumentacja ShipX 1.9.0 na przeglądarkanej stronie NIE dokumentuje `custom_attributes`. Może być feature specyficzny dla `organization_id` (custom pola per klient).

Weryfikacja: sprawdzić w `curl` przeciwko sandboxowi ShipX czy field jest akceptowany (integration test opt-in). Alternatywnie: pobrać Czek z **InPost support** czy `custom_attributes.sending_method` to publiczny kontrakt.

Fix (F-I1): dodać integration test `test_inpost_shipx_contract.py` (opt-in, `@pytest.mark.integration`) który waliduje POST przeciwko sandboxowi.

1.3 OK LOW — `target_point` dla paczkomatu

Kod używa `custom_attributes.target_point: <locker_id>`. Standardowy pattern ShipX dla paczkomatu to `receiver.address.line1 = "PACZKOMAT_ID"` LUB `custom_attributes.target_point`. Oba są używane w różnych integracjach — nasze użycie **prawdopodobnie poprawne** (potwierdza działanie w prod), ale warto zweryfikować.

Fix (F-I2): włącz w F-I1 integration test też paczkomatowy flow.

2. Endpointy — inwentaryzacja

#	Method	Path	Idempotent	Retry safe	Użytkowane?
1	POST	<code>/v1/organizations/{id}/shipments</code>	X	X	OK (paczkomat + kurier)
2	POST	<code>/v1/organizations/{id}/dispatch_orders</code>	X	X	OK
3	DELETE	<code>/v1/shipments/{id}</code>	Tak	OK	MED metoda istnieje, ale brak routera (<code>webhooks.py</code>) — Fix F-I3
4	DELETE	<code>/v1/organizations/{id}/dispatch_orders/{id}</code>	Tak	OK	MED j.w.
5	GET	<code>/v1/shipments/{id}/label</code>	Tak	OK	OK (get_label)

Brakuje 2 routerów DELETE (zgodne z deklaracją w naszym `webhooks.py:1-11` docstringu, ale nie zaimplementowane):

- `DELETE /inpost/shipments/{id}` → `client.cancel_shipment(id)`
- `DELETE /inpost/dispatch_orders/{id}` → `client.cancel_dispatch_order(id)`

3. Antypatery — ruff strict (inpost.py)

Reguła	Ile	Linie	Severity	Opis
TRY003	7	202, 252, 265, 268, 279, 282, 293	OK low	Długie messages w <code>raise InPostError(...)</code> . Fix: struct wyjątków (Fix I5) lub whitelist.
EM102	7	j.w.	OK low	f-string literal w raise. Powiązane z TRY003.
RUF003	4	72, 80, 87, 94	OK low	Ambiguous chars <code>?/OK</code> w KOMENTARZACH. False positive — to dokumentacja. Whitelist.
PLR2004	3	264, 278	MED medium	Magic value <code>422</code> . Fix: <code>HTTP_UNPROCESSABLE_ENTITY = 422</code> (do cross-cutting).

4. Idempotency

4.1 Klient

- **POST shipments/dispatch_orders NIE są idempotent.** Każdy call = nowa przesyłka (opłata!).
- **DELETE endpointy są idempotent:** 200 przy sukcesie, 422 przy "already dispatched" (semantyczne "no-op czy conflict").

4.2 Braki

- ShipX nie oferuje `Idempotency-Key` header — dedup musi być po naszej stronie (`shipping_store` guard: nie POST-uj jeśli `courier_draft_id` już set na draft).
- Obecnie: `_run_inpost` (`webhooks.py:243+`) nie sprawdza czy `draft.courier_draft_id` już istnieje — retry z UI może utworzyć duplikat.

Fix (F-I3): w `_run_inpost` sprawdzać przy wejściu:

```
if draft.get("courier_draft_id"):
    raise HTTPException(409, "Shipment already created – DELETE first if you want to recreate")
```

5. Exception mapping — **CRIT** CRITICAL problem

5.1 `InPostError(Exception)` — poza hierarchią

Bug #I2 (CRITICAL — antypater) — `inpost.py:107-108`:

```
class InPostError(Exception):
    pass
```

Problemy:

1. **Nie dziedziczy z `ZdrovenaShippingError`** → w routerze `except ZdrovenaShippingError` **NIE** złapie `InPostError` → wpadnie do bardziej ogólnego `except Exception` (BLE001).
2. **Brak rozróżnienia** auth vs business vs transient error — wszystkie mapuje na 1 klasę.
3. **Brak strukturalnych atrybutów** (courier, status, action) jak w `CourierBusinessError`.

Fix (F-I4 CRITICAL): refactor `inpost.py` na wzór `fakturownia.py`:

```
from zdrovena.common.shipping_exceptions import (
    CourierAuthError,
    CourierBusinessError,
    CourierServerError,
    CourierTimeoutError,
    CourierConnectionError,
    InPostAuthError,          # istnieje już
    InPostLockerUnavailableError, # istnieje już
    InPostInvalidServiceError, # istnieje już
)

class InPostClient:
    def _parse_response(self, resp, *, method, path):
        status = resp.status_code
        if 200 <= status < 300:
            return resp.json() if status != 204 else None
        body = resp.text[:300]
        if status in (401, 403):
            raise InPostAuthError(detail=body)
        if status == 422:
            # ShipX validation error – mapować na business
            raise CourierBusinessError(courier="inpost", detail=body, action=f"{method} {path}")
        if 400 <= status < 500:
            raise CourierBusinessError(courier="inpost", detail=body, action=f"{method} {path}")
        if status >= 500:
            raise CourierServerError(courier="inpost", status=status)
```

Zamiast `raise InPostError(...)` — używać typowanej hierarchii.

Backward compat: pozostawić `InPostError = ZdrovenaShippingError` (deprecated alias) na 1 release, dodać deprecation warning.

5.2 Timeout/connection nieobsługiwane

Bug #I3 (MEDIUM) — `inpost.py:198-212, 250-255, 261-271, 275-285, 289-294` używa `self._session.post/get/delete` bez `try/except`. Jeśli `requests.Timeout` lub `requests.ConnectionError` — propaguje raw wyjątek. Powinno być zamapowane na `CourierTimeoutError` / `CourierConnectionError`.

Fix (F-I5): dodać `_request()` helper (jak w `fakturownia.py:89-113`) z `try/except` na `Timeout/ConnectionError`.

6. Logging

6.1 Stan obecny

- `_post_shipment` (201): `logger.info("InPost shipment created: id=%s tracking=%s service=%s", ...)` OK
- `create_dispatch_order` (232): `logger.info("InPost dispatch order created: id=%s", ...)` OK
- `cancel_shipment` (250): `logger.info("InPost shipment cancelled: id=%s", ...)` OK
- `cancel_dispatch_order` (264): `logger.info("InPost dispatch order cancelled: id=%s", ...)` OK
- Brak logów WARNING/ERROR — wszystkie błędy propagują jako wyjątek. Router loguje sam.

Ocena: dobrze. Klient jest cichy w error path (raise), głośny w success path (info). +

6.2 Rekomendacja

Dodać `logger.debug` na req/resp payload (z redakcją PII — email, telefon receivera). Pomocne przy troubleshootingu.

7. Lista fixów do finalnego PR (InPost)

Blok A — Exception refactor (CRITICAL):

- ☐ **F-I4:** `inpost.py:107` — usunąć `InPostError(Exception)`. Refactor `_post_shipment`, `cancel_shipment`, `cancel_dispatch_order`, `get_label` na typowaną hierarchię (`InPostAuthError`, `CourierBusinessError`, `CourierServerError`). Alias `InPostError = ZdrovenaShippingError` z `DeprecationWarning`.
- ☐ **F-I5:** Dodać `_request()` helper z mapowaniem `requests.Timeout` → `CourierTimeoutError`, `requests.ConnectionError` → `CourierConnectionError`.

Blok B — Cancel endpoints (feature completion):

- ☐ **F-I3:** Dodać w `webhooks.py`:
 - `DELETE /inpost/shipments/{id}` → `client.cancel_shipment(id)` → 204 lub 409 (już dispatched)

- `DELETE /inpost/dispatch_orders/{id}` → `client.cancel_dispatch_order(id)` → 204 lub 409
- `POST /shipping/drafts/{id}/execute` — guard `if draft.get("courier_draft_id"): raise 409`

• ☐

F-I6: usuń z docstringa `webhooks.py:1-11` obietnice DELETE albo poczekaj po F-I3.

Blok C — Contract validation (MEDIUM):

• ☐

F-I1: Integration test `tests/integration/test_inpost_shipx_contract.py` (opt-in, `@pytest.mark.integration`) — sandbox POST + assert response schema.

• ☐

F-I7: Rozwiązać `custom_attributes.sending_method` / `target_point` — potwierdzenie od InPost support LUB test na sandboxie.

Blok D — Cosmetics (low, do cross-cutting):

• ☐

F-I8: `inpost.py:264, 278` — magic 422 → stała.

• ☐

F-I9: `inpost.py:72, 80, 87, 94` — RUF003 (? / OK) — whitelist w `pyproject.toml` (to dokumentacja, nie kod).

Estymacja: ~250 linii kodu produkcyjnego + ~150 linii testów + 1 nowy integration test file.

Załączniki

Źródła:

- [InPost ShipX 1.9.0 Shipment](#)
- Wewnętrzna dokumentacja: `zdrovena/common/shipping_exceptions.py` (hierarchia bazowa)

Audyt techniczny — Apaczka

Zakres: `zdrovena/common/apaczka.py` (190 linii) — klient Apaczka Web API v2 z HMAC-SHA256 + kod webhooks.py w sekcji Apaczka.

Metodologia: kontraktowy audyt względem [Dokumentacja Web API ver. 2 \(Apaczka\)](#) + `ruff strict` + `pyright` + manual review.

Wynik pyright: OK 0 errors. **Wynik ruff strict:** OK 10+ znalezisk (głównie stylowe: TRY003, EM102, BLE001 w cache fallback).

Ogólna ocena: HMAC signature format i endpointy zgodne z dokumentacją. Kluczowe problemy identyczne jak w InPost: `ApaczkaError(Exception)` poza hierarchią + brak mapowania na retry-able.

1. Kontraktowe niezgodności z Apaczka API v2

1.1 OK Zgodne z dokumentacją

Aspekt	Nasz kod	Dokumentacja	Status
Base URL	<code>https://www.apaczka.pl/api/v2</code> (<code>apaczka.py:22</code>)	Zgodne	OK
Endpoint call: <code>POST {base}/{endpoint}/</code>	<code>f"_{BASE}/{endpoint}/"</code> (<code>apaczka.py:55</code>)	"Wszystkie dane należy kierować na odpowiedni endpoint..."	OK
Body params	<code>app_id</code> , <code>request</code> , <code>expires</code> , <code>signature</code>	Zgodne (4 wymagane pola)	OK
Content-Type	<code>data=body</code> (form-encoded, requests default)	Zgodne (POST)	OK
HMAC signature	<code>HMAC-SHA256({app_id}:{endpoint}:{request_json}:{expires}, secret)</code> (<code>apaczka.py:32-42</code>)	"signature musi być wygenerowana na podstawie App ID, nazwy endpoint'u, danych w request oraz daty wygaśnięcia... HMAC + SHA256, klucz App Secret"	OK
<code>expires</code> max	<code>time.time() + 1800</code> (30 min) (<code>apaczka.py:34</code>)	"Maksymalna ważność request'u to 30 minut"	OK
Response envelope	<code>result.get("status")</code> sprawdza (200 = OK) (<code>apaczka.py:59</code>)	Response wrapper z <code>status</code>	OK
Endpoints użytkowane	<code>service_structure</code> , <code>order_send</code> , <code>order_cancel</code> , <code>waybill</code>	Wszystkie 4 udokumentowane	OK

Zero bugów kontraktowych w kliencie Apaczka. +

1.2 MED MEDIUM — signature format vs `json.dumps` separators

Bug #A1 (MEDIUM) — `apaczka.py:33`:

```
request_json = json.dumps(data, separators=(",", ":"))
```

Dokumentacja Apaczka mówi że `request` to "zestaw wymaganych danych zapisanych w strukturze JSON" — **nie precyzuje** czy signature ma być liczony na compact JSON, na normalized JSON, ani jak encodować polskie znaki.

Ryzyko: jeśli Apaczka zmieni parsing (obecnie akceptuje compact JSON i signature obliczony na tym samym compact JSON — dowód: kod działa), to zmiana strategii separatorów po naszej stronie **zepsuje signature**. Trzeba mieć test regresyjny.

Fix (F-A1): dodać unit test `test_apaczka_sign_signature` z hard-codowanymi:

- `app_id`, `secret`, `endpoint`, `data`, `expires`
- oczekiwany signature (obliczony ręcznie/z Postman) Chroni przed regresją przy zmianie `separators`, `ensure_ascii`, `sort_keys`.

1.3 OK LOW — Brak `is_zebra` w request

Dokumentacja wspomina: "Parametr `is_zebra` jest opcjonalny. W przypadku jego nie podania etykieta będzie wygenerowana zgodnie z ustawieniami konta."

Nasz kod NIE ustawia `is_zebra`. **OK** — używamy default (Zebra vs A4 = konfiguracja konta).

2. Endpointy — inwentaryzacja

#	Method	Path	Idempotent	Retry safe	Użytkowane?
1	POST	<code>/api/v2/service_structure/</code>	Tak	OK	OK (cache 23h)
2	POST	<code>/api/v2/order_send/</code>	X	X	OK
3	POST	<code>/api/v2/order_cancel/</code>	Tak (idempotent)	OK	OK metoda w kliencie, brak routera — Fix F-A3
4	POST	<code>/api/v2/waybill/</code>	Tak	OK	OK (get_label)

Brakuje 1 routera DELETE (deklaracja w `webhooks.py:1-11` docstringu):

- `DELETE /apaczka/orders/{id}` → `client.cancel_shipment(id)`

3. Antypaterny — ruff strict (`apaczka.py`)

Reguła	Ile	Linie	Severity	Opis
BLE001	2	75, 92	MED medium	<code>except Exception:</code> <code>pass</code> w cache read (linia 75) i <code>logger.warning</code> bez <code>exception</code> w cache write (linia 92). Fix: <code>except (KeyError, ValueError, json.JSONDecodeError, StorageError)</code> explicit.
TRY003	3	57, 60, 188	OK low	<code>raise</code> <code>ApaczkaError(f"...")</code> . Fix: wspólny z InPost (F-A4).
EM102	3	j.w.	OK low	j.w.
PLC0415	3	67, 86, 183	OK low	<code>import io</code> , <code>import base64</code> w środku funkcji. Fix: top-level.

4. Idempotency

4.1 Klient

- `service_structure` — cachowany 23h (dobrze!). OK
- `order_send` — NIE idempotent. Każdy call = nowe zamówienie = opłata!
- `order_cancel` — Apaczka akceptuje wielokrotny cancel na tym samym `order_id` (idempotent).
- `waybill` — GET semantics przez POST (nietypowe API, ale idempotent).

4.2 Braki

Bug #A2 (MEDIUM) — brak guard w `_run_apaczka` przed POST `order_send`. Jeśli draft ma już `courier_draft_id` (zamówienie utworzone) i user kliknie "execute" ponownie → duplikat zamówienia + duplikat opłaty.

Fix (F-A2): analogicznie do InPost F-I3 — sprawdzać `draft.courier_draft_id` przed `_run_apaczka`.

4.3 Cache invalidation

Cache `service_structure` ma TTL 23h. **OK**, ale brak invalidation przy błędzie (np. gdy Apaczka doda nową usługę i my używamy 22h-stały cache). Rekomendacja: cache bust przy `ApaczkaError` z komunikatem "invalid service_id".

5. Exception mapping — **CRIT** CRITICAL problem

5.1 `ApaczkaError(Exception)` — poza hierarchią

Bug #A3 (CRITICAL — antypater) — `apaczka.py:28-29`:

```
class ApaczkaError(Exception):
    pass
```

Problem identyczny jak w InPost — nie dziedziczy z `ZdrovenaShippingError`, brak rozróżnienia typów błędów.

Dodatkowo hierarchia W `shipping_exceptions.py` **MA** już Apaczka-specific errors:

- `ApaczkaSignatureError(CourierAuthError)` (linia 172)
- `ApaczkaInsufficientBalanceError(CourierAuthError)` (linia 192)
- `ApaczkaServiceUnavailableError(CourierBusinessError)` (linia 230)

Ale klient ich **NIE UŻYWA!** `_call` (linia 53-61) rzuca tylko `ApaczkaError`.

Fix (F-A4 CRITICAL): refactor `_call` na typowaną hierarchię:

```
def _call(self, endpoint: str, data: dict[str, Any]) -> dict[str, Any]:
    body = _sign(self._app_id, self._secret, endpoint, data)
    try:
        resp = self._session.post(f"{_BASE}/{endpoint}/", data=body, timeout=_TIMEOUT)
    except requests.Timeout as e:
        raise CourierTimeoutError(courier="apaczka", action=endpoint) from e
    except requests.ConnectionError as e:
        raise CourierConnectionError(courier="apaczka", detail=str(e)) from e

    if not resp.ok:
        if resp.status_code >= 500:
            raise CourierServerError(courier="apaczka", status=resp.status_code)
        raise CourierBusinessError(courier="apaczka",
                                   detail=f"{resp.status_code}: {resp.text[:200]}",
                                   action=endpoint)

    result = resp.json()
    if result.get("status") != 200:
        message = result.get("message", "")
        if "signature" in message.lower():
            raise ApaczkaSignatureError(detail=message)
        if "insufficient" in message.lower() or "saldo" in message.lower():
            raise ApaczkaInsufficientBalanceError(detail=message)
        if "service" in message.lower():
            raise ApaczkaServiceUnavailableError(detail=message)
        raise CourierBusinessError(courier="apaczka", detail=message, action=endpoint)
    return result
```

Backward compat: `ApaczkaError = ZdrovenaShippingError` (deprecated alias).

5.2 Cache read `except Exception: pass`

Bug #A4 (MEDIUM) — `apaczka.py:75-76`:

```
except Exception:
    pass
```

Silent fail — jeśli storage service jest niedostępny, kontynuujemy i robimy fresh API call. **W praktyce to jest OK** (fallback do świeżego call), ale:

- Nie logujemy że cache miss z powodu błędu.
- Nie mierzymy jak często storage service pada.

Fix (F-A5): `except (KeyError, ValueError, json.JSONDecodeError) as exc: logger.debug("Apaczka cache miss: %s", exc)`. Dla `StorageError` osobno: `logger.warning`.

6. Logging

6.1 Stan obecny

- `_call` sukces: brak logu na sukces (INFO). Powinien być w `_call` (audit trail).
- `create_shipment` sukces: `logger.info("Apaczka shipment created: order_id=%s reference=%s", ...)` **OK**
- `cancel_shipment` sukces: `logger.info("Apaczka shipment cancelled: order_id=%s", ...)` **OK**
- Cache miss: `logger.info("Fetching Apaczka service_structure (cache miss)")` **OK**
- Cache write fail: `logger.warning("Failed to cache Apaczka service_structure: %s", exc)` — **!** powinno być `.exception` (TRY400 by wykryło, ale reguła jest wyłączona ANN).
- Cache read fail: **cichy** (bug #A4).

6.2 Rekomendacja

- Dodać `logger.info` na happy path `_call` (audit trail: endpoint + response status).
- Zamienić `logger.warning(..., exc)` w `apaczka.py:93` na `.exception()`.
- Dodać `logger.exception` w cache read fallback.

7. Lista fixów do finalnego PR (Apaczka)

Blok A — Exception refactor (CRITICAL):

• ☐

F-A4: `apaczka.py:28` — usuń `ApaczkaError(Exception)`. Refactor `_call` na typowaną hierarchię (`ApaczkaSignatureError`, `ApaczkaInsufficientBalanceError`, `ApaczkaServiceUnavailableError`, `CourierBusinessError`, `CourierServerError`, `CourierTimeoutError`, `CourierConnectionError`). Alias `ApaczkaError = ZdrovenaShippingError` z `DeprecationWarning`.

Blok B — Cancel endpoint (feature completion):

• ☐

F-A3: Dodać w `webhooks.py`:

- `DELETE /apaczka/orders/{id}` → `client.cancel_shipment(id)` → 204 (Apaczka jest idempotent, więc bez 422)
- `POST /shipping/drafts/{id}/execute` — guard `if draft.get("courier_draft_id"): raise 409`

Blok C — Contract & idempotency (MEDIUM):

- ☐ **F-A1:** Unit test `test_apaczka_sign_signature` — hardcoded fixture, chroni signature format.
- ☐ **F-A2:** `_run_apaczka` — guard przed re-POST `order_send`.
- ☐ **F-A5:** `apaczka.py:75` — explicit except types + `logger.debug/warning` (nie pass).

Blok D — Cosmetics (low):

- ☐ **F-A6:** `apaczka.py:67,86,183` — `import io`, `import base64` na top-level.
- ☐ **F-A7:** `apaczka.py:93` — `logger.warning(...)` → `.exception()` (wraz z TRY400 sweep).

Estymacja: ~150 linii kodu produkcyjnego + ~100 linii testów.

Załączniki

Źródła:

- [Dokumentacja Web API ver. 2 - Apaczka](#) (2022-08-22)
- [Apaczka Push Tracking API PDF](#) (dla webhooków tracking-back — obecnie nie używamy)
- Wewnętrzna dokumentacja: `zdrovena/common/shipping_exceptions.py` (Apaczka-specific error classes 172-197)

Audyt cross-cutting — antypaternalny wspólny dla wszystkich integracji

Zakres: wzorce projektowe i antypaternalny, które pojawiają się w więcej niż jednej integracji. Ten raport konsoliduje findings z Allegro, Fakturownia, Shopify, InPost i Apaczka.

Metodologia: agregacja `ruff strict` + `pyright` + manual review z §3 raportów 01-05.

1. Hierarchia wyjątków — potrzeba unifikacji

1.1 Obecny stan

`zdrovena/common/shipping_exceptions.py` definiuje bogatą hierarchię 30+ klas:

```
ZdrovenaError
├─ ZdrovenaShippingError
│   ├── ShopifyPayloadError → 400 (walidacja Shopify webhook payload)
│   ├── CourierAuthError → 502 (auth do dostawcy)
│   │   ├── AllegroAuthError
│   │   ├── InPostAuthError
│   │   ├── ApaczkaSignatureError
│   │   └─ ApaczkaInsufficientBalanceError
│   ├── CourierBusinessError → 502 (4xx z API dostawcy)
│   │   ├── AllegroBusinessError
│   │   │   └─ AllegroCommandPending
│   │   ├── InPostLockerUnavailableError
│   │   ├── InPostInvalidServiceError
│   │   ├── ApaczkaServiceUnavailableError
│   │   ├── PickupSlotUnavailableError
│   │   └─ AddressGeocodingError
│   ├── CourierTransientError → 502 (retry-able)
│   │   ├── CourierTimeoutError
│   │   ├── CourierConnectionError
│   │   └─ CourierServerError
│   ├── CancellationError → 409
│   │   ├── ShipmentAlreadyDispatchedError
│   │   ├── DispatchAlreadyAcceptedError
│   │   └─ MissingDispatchIdError
│   ├── FakturowniaAuthError → CourierAuthError
│   ├── FakturowniaBusinessError → CourierBusinessError
│   └─ FakturowniaServerError → CourierServerError
```

Ocena: hierarchia jest **kompletna i dobrze zaprojektowana**. Semantyka: Auth = alert, Business = 4xx, Transient = retry-able.

1.2 CRIT CRITICAL — klienci NIE UŻYWAJĄ tej hierarchii konsekwentnie

Klient	Używa hierarchii?	Rzuca
<code>allegro.py</code>	OK (fully)	AllegroAuthError, AllegroBusinessError, CourierServerError

Klient	Używa hierarchii?	Rzuca
<code>fakturownia.py</code>	OK (fully)	FakturowniaAuthError, FakturowniaBusinessError, FakturowniaServerError, CourierTimeoutError, CourierConnectionError
<code>inpost.py</code>	CRIT NIE	<code>InPostError(Exception)</code> poza hierarchią
<code>apaczka.py</code>	CRIT NIE	<code>ApaczkaError(Exception)</code> poza hierarchią

Konsekwencja: router `webhooks.py` łapie `except Exception` (BLE001) bo nie może polegać na `except ZdrovenaShippingError` — bo InPost/Apaczka rzucają **poza** tą hierarchią.

Fix wspólny (F-X1 CRITICAL) — Blok A w Raportach InPost (F-I4) + Apaczka (F-A4):

1. Usuń `InPostError(Exception)` i `ApaczkaError(Exception)`.
2. Refactor klientów na typowaną hierarchię (istniejące klasy w `shipping_exceptions.py`).
3. Alias `InPostError = ApaczkaError = ZdrovenaShippingError` (deprecated, 1 release).
4. W routerze zamień `except Exception` → `except ZdrovenaShippingError` + granular handling.

2. `except Exception` (BLE001) — 12 miejsc w kodzie

2.1 Lokalizacje

Plik	Linie	Powód
<code>allegro_poller.py</code>	56, 67, 87, 100	Poller: łapie wszystko żeby nie crashować cyklu
<code>fakturownia_patcher.py</code>	100, 113, 132	Patcher: j.w.
<code>webhooks.py</code>	100, 187, 321, 528, 732, 869, 934, 1039	Routery: łapią all → 500
<code>apaczka.py</code>	75, 92	Cache read/write fallback

2.2 Fix wspólny (F-X2)

```
# BAD
try:
    do_work()
except Exception as exc:
    logger.error("failed: %s", exc)

# GOOD
try:
    do_work()
except (CourierTransientError, CourierAuthError, CourierBusinessError):
    logger.exception("Known error class")
    raise # lub obsłuż kontekstowo
```

```
except Exception:
    logger.exception("Unexpected error – please add to hierarchy")
    raise
```

Pattern **guard rails + typed catches**:

- `except (Typed1, Typed2)` — obsłuż każdy explicit
- `except Exception` — tylko jako ostatni resort, ZAWSZE z `.exception()` i re-raise / metric

Whitelist: routery mogą łąpać `Exception` przed konwersją na `HTTPException(500)`, ale ZAWSZE z `.exception()`.

3. `logging.error(..., exc)` (TRY400) — 12 miejsc

Zamiana na `.exception(...)` — traceback bierze się automatycznie z `exc_info=True`.

3.1 Lokalizacje

Plik	Linie
<code>allegro_poller.py</code>	57, 68, 88
<code>fakturownia_patcher.py</code>	101, 114, 133
<code>webhooks.py</code>	188, 870, 935, 1040
<code>apaczka.py</code>	93 (jest <code>.warning</code> , docelowo <code>.exception</code>)

3.2 Fix wspólny (F-X3)

```
# BAD
logger.error("Allegro list_orders failed: %s", exc)

# GOOD
logger.exception("Allegro list_orders failed")
# → automatycznie dołącza stack trace
```

Reguła: każdy `except` musi używać `.exception()` (chyba że explicit `.debug()` / `.warning()` bez potrzeby traceback).

4. Magic HTTP status codes (PLR2004) — 15 miejsc

4.1 Lokalizacje

Plik	Wartości
<code>allegro.py</code>	400, 500, 204 (linie 168, 173, 182, 191)
<code>fakturownia.py</code>	200, 300, 204, 500, 600 (linia 118, 133)
<code>inpost.py</code>	422 (linia 264, 278)
<code>webhooks.py</code>	wiele (400, 401, 404, 409, 502, 503, ...)

4.2 Fix wspólny (F-X4)

Stworzyć `zdrovena/common/http_status.py`:

```
# Successful
HTTP_OK = 200
HTTP_NO_CONTENT = 204

# Client errors
HTTP_BAD_REQUEST = 400
HTTP_UNAUTHORIZED = 401
HTTP_FORBIDDEN = 403
HTTP_NOT_FOUND = 404
HTTP_CONFLICT = 409
HTTP_UNPROCESSABLE_ENTITY = 422

# Server errors
HTTP_INTERNAL_SERVER_ERROR = 500
HTTP_BAD_GATEWAY = 502
HTTP_SERVICE_UNAVAILABLE = 503

# Ranges
HTTP_STATUS_SUCCESS = range(200, 300)
HTTP_STATUS_CLIENT_ERROR = range(400, 500)
HTTP_STATUS_SERVER_ERROR = range(500, 600)
```

Alternatywa: użyć `http.HTTPStatus` z stdlib (enum). Preferowane, bo:

- `HTTPStatus.NO_CONTENT.value == 204`
- `HTTPStatus.OK` czytelne w kodzie
- Zero maintenance ze strony naszej strony

Fix (F-X4): zamień wszystkie liczby-magic na `HTTPStatus.<NAME>.value` LUB własne stałe.

5. Bezpieczeństwo — spotkane antypaternalny

5.1 Idempotency — wszystkie POST-y

Wszystkie 5 integracji ma non-idempotent POST endpointy które **każdy call kosztuje** (opłata za przesyłkę, opłata za command Allegro, itd.). Brak `Idempotency-Key` header na naszych publicznych endpointach.

Fix (F-X5):

- Wszystkie publiczne POST `/allegro/...`, `/shipping/drafts/{id}/execute`, `/inpost/...`, `/apaczka/...` powinny wymagać nagłówka `Idempotency-Key: <uuid>`.
- Dedup w Redis TTL 30 min. Cache klucza + odpowiedzi. Duplikat → zwróć cached response.
- Alternatywnie: guardy w `shipping_store` (jest już `shopify_order_id` dedup w draftach).

5.2 Silent exceptions w idempotency check

`webhooks.py:719-734` (`_is_duplicate_webhook`) — `except Exception: pass` → jeśli storage padnie, tworzymy duplikaty draftów.

Fix (F-X6): Fail-fast w idempotency checks. Silent fallback tylko dla non-critical work (np. cache write).

5.3 HMAC bypass (Shopify)

`ALLOW_UNSIGNED_SHOPIFY_WEBHOOKS=1` przepuszcza niesignowane webhooks. **Do usunięcia w prod** — patrz Raport Shopify F-S2.

6. Testowalność — braki wspólne

6.1 Brak testów kontraktowych

Wszystkie 4 zewnętrzne integracje (Allegro, Fakturownia, InPost, Apaczka) mają **jednostkowe testy z mocks**, ale **zero testów kontraktowych** przeciwko realnym sandboxom.

Konsekwencja: nasze mocki mogą dryftować od kontraktu. Bug #1 (invoiceNumber vs number) był ukryty właśnie dlatego — 15 testów z fikcyjnym `"number"` przechodziło.

Fix wspólny (F-X7):

- Nowa struktura: `tests/integration/test_<integration>_contract.py`
- Markery: `@pytest.mark.integration` (opt-in, nie w default fast gate)
- Runbook: cotygodniowy nightly run w CI przeciwko sandboxom
- Fixtures: przechowywać w `tests/integration/fixtures/` (JSON responses z realnych API)

6.2 Testy fixture drift

Fixture `docs/audit/fixtures/allegro_get_invoices.json` (z Fazy 2) jako "golden master" — testy powinny go używać, nie fikcyjnych struktur.

Fix (F-X8): przy każdym integration test run, jeśli response format się zmieni → alert, wygeneruj diff, wymagaj human review.

7. Logging standard (docelowy)

7.1 Convention

```
# Success (audit trail)
logger.info("%s: created %s (id=%s)", carrier, action, ext_id)

# Retry-able (transient)
logger.warning("%s: transient error, will retry: %s", carrier, exc)

# Fatal — with traceback
logger.exception("%s: unexpected error", carrier) # exc_info auto
```

```
# Security event
logger.warning("Shopify HMAC mismatch – rejected. correlation_id=%s", webhook_id)

# Debug (protocol tracing) – with PII redaction
logger.debug("Apaczka payload: %s", redact_pii(payload))
```

7.2 Structured fields obowiązkowe

Każdy log w integracji MUSI zawierać:

- `carrier` (allegro/inpost/apaczka/fakturownia/shopify)
- `action` (list_orders/create_shipment/order_send/...)
- `external_id` (jeśli mamy — order_id, invoice_id, tracking_number)
- `correlation_id` (X-Shopify-Webhook-Id lub UUID z Idempotency-Key)

7.3 PII redaction

Payloady w `logger.debug` mogą zawierać email/telefon receivera. **Zawsze redaktować** przed logowaniem (regex email/phone → `***`).

8. Docstring vs kod — obietnice bez pokrycia

8.1 `webhooks.py:1-11` mówi:

POST /webhooks/shopify/order-created	– <code>OK</code> istnieje
GET /shipping/drafts	– <code>OK</code> istnieje
GET /shipping/drafts/{id}/label	– <code>OK</code> istnieje
POST /shipping/drafts/{id}/execute	– <code>OK</code> istnieje
POST /shipping/drafts/{id}/pickup (Apaczka)	– <code>OK</code> istnieje
DELETE /shipping/drafts/{id}/shipment	– <code>CRIT</code> NIE ISTNIEJE
DELETE /shipping/drafts/{id}/dispatch	– <code>CRIT</code> NIE ISTNIEJE

Fix (F-X9) — decyzja usera: dodaj DELETE endpointy (nie usuwaj z docstringa):

- `DELETE /allegro/drafts/{id}/shipment` (Fix F6 z Raportu Allegro)
- `DELETE /allegro/drafts/{id}/dispatch` (Fix F7 z Raportu Allegro)
- `DELETE /inpost/shipments/{id}` (Fix F-I3 z Raportu InPost)
- `DELETE /inpost/dispatch_orders/{id}` (Fix F-I3 z Raportu InPost)
- `DELETE /apaczka/orders/{id}` (Fix F-A3 z Raportu Apaczka)

9. Podsumowanie fixów do finalnego PR

9.1 CRITICAL (blokujące feature)

ID	Integracja	Opis	Szacunek LoC
F1	Allegro	<code>invoiceNumber</code> fix + 15 testów	20 + 60
F2-F5	Allegro	4 bugi Ship with Allegro <code>create-commands</code> payload	80 + 100
F6-F7	Allegro	DELETE endpointy + cancel methods	80 + 60
F-S1	Shopify	<code>X-Shopify-Webhook-Id</code> dedup table	70 + 100
F-S2	Shopify	Usuń <code>ALLOW_UNSIGNED_SHOPIFY_WEBHOOKS</code> z prod	10 + 20
F-S6	Shopify	Whitelist <code>X-Shopify-Topic</code> i <code>X-Shopify-Shop-Domain</code>	30 + 40
F-I4	InPost	Refactor <code>InPostError(Exception)</code> → typowana hierarchia	100 + 80
F-A4	Apaczka	Refactor <code>ApaczkaError(Exception)</code> → typowana hierarchia	80 + 60

9.2 MEDIUM (jakość / obserwabylność)

ID	Integracja	Opis	Szacunek LoC
F-X1	wspólny	Zunifikuj <code>except ZdrovenaShippingError</code> w routerze	50
F-X2	wspólny	Typed catches (BLE001) — 12 miejsc	100
F-X3	wspólny	<code>.error(..., exc)</code> → <code>.exception()</code> — 12 miejsc	20
F-X4	wspólny	Magic HTTP codes → <code>HTTPStatus.<NAME>.value</code>	40
F-X5	wspólny	<code>Idempotency-Key</code> header dla POST + Redis dedup	80 + 100
F-S3, F-A2	Shopify + Apaczka	Race protection w duplikatach draftów	40
F-S5	Shopify	<code>get_draft_by_shopify_order_id</code> (indexed)	30 + 30
F-I3, F-I5	InPost	Cancel endpointy + <code>_request()</code> helper	100 + 80
F-I1, F-I7	InPost	Integration test <code>test_inpost_shipx_contract</code>	80

ID	Integracja	Opis	Szacunek LoC
F-A1, F-A3	Apaczka	Cancel endpoint + sign test	40 + 40
F-A5	Apaczka	Cache read explicit types	15
FA1-FA3	Fakturownia	Integration test + deep copy + <code>id</code> assertion	60 + 40

9.3 LOW (cosmetic, cross-cutting)

ID	Opis	Szacunek LoC
F-X4	Magic values → stałe (cały codebase)	40
F15	Usuń 6 martwych Allegro methods	-100 (delete)
RUF002, RUF003	Unicode chars w docstringach (<code>×</code> , <code>?</code> , <code>OK</code>)	5
PYI041	<code>`int`</code>	<code>float</code> → <code>float`</code>
PLC0415	Imports w środku funkcji (~10 miejsc)	20
ERA001	Commented-out code	5
EM101/EM102	Whitelist w <code>pyproject.toml</code> (rekomendowane) LUB refactor	15

9.4 Skala PR

Suma: ~1400 linii kodu produkcyjnego + ~800 linii testów.

Struktura commits w PR (rekomendowana):

- `feat(shipping-errors): unify InPost/Apaczka into ZdrovenaShippingError hierarchy` (F-I4 + F-A4 + F-X1)
- `fix(allegro): invoiceNumber field, ship-with-allegro schema, sender/receiver blocks` (F1-F5)
- `feat(allegro): add DELETE endpoints for shipment/dispatch cancel` (F6-F7)
- `feat(inpost,apaczka): add DELETE cancel routers` (F-I3 + F-A3)
- `fix(shopify): webhook-id dedup, remove unsigned bypass, topic whitelist` (F-S1, F-S2, F-S6)
- `feat(shipping): add Idempotency-Key on public POST endpoints` (F-X5)
- `chore(logging): replace .error(..., exc) with .exception()` (F-X3)
- `chore(types): explicit exception catches instead of blind Exception` (F-X2)
- `chore(constants): HTTPStatus enum for magic status codes` (F-X4)
- `test(integration): contract tests for Allegro/Fakturownia/InPost/Apaczka` (F-X7, F-I1, F-A1, FA1)
- `chore(cleanup): remove dead Allegro methods` (F15)

Załączniki

- Raporty pełne per integracja: [01-allegro.md](#), [02-fakturownia.md](#), [03-shopify.md](#), [04-inpost.md](#), [05-apaczka.md](#)
- Executive summary: [00-summary.md](#)
- Fixtures: [fixtures/](#) — 3 pliki JSON z realnymi kontraktami Allegro